

Asymptotic Analysis

Recursive Fibonacci

- Let $T(n)$ be the number of operations done by $Fib(n)$
- $T(0) = T(1) = 2$ for checking if statement
- $\forall n \geq 2, T(n) = T(n-1) + T(n-2) + 8$ for the extra checking and calls
- So $T(n) \geq Fib(n)$
- $Fib(n) = Fib(n-1) + Fib(n-2), Fib(n) \geq 2 \cdot Fib(n-2)$
- Doubles every two terms, so it doubles $\lceil \frac{n-2}{2} \rceil$ times and $T(n) \geq Fib(n) \geq 2^{\frac{n-2}{2}}$

Big O Notation

- $f \in O(g) : \exists c > 0, n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq cg(n)$, g is an upper bound on f , also written as $f(n) = O(g(n))$
- Multiple c and n_0 could work for the same case
- $f \in \Omega(g) : \exists c > 0, n_0 > 0, \forall n \geq n_0, 0 \leq cg(n) \leq f(n)$, g is a lower bound on f
- $f \in \Theta(g) : \text{tight bound, } \Theta(g) = O(g) \cap \Omega(g)$
- $f \in o(g) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < cg(n)$, strict upper bound
- $f \in \omega(g) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq cg(n) < f(n)$, strict lower bound

Limits

- $\lim_{n \rightarrow \infty} (\frac{f(n)}{g(n)}) = 0 \Rightarrow f(n) \in o(g(n))$
- $\lim_{n \rightarrow \infty} (\frac{f(n)}{g(n)}) < \infty \Rightarrow f(n) \in O(g(n))$
- $0 < \lim_{n \rightarrow \infty} (\frac{f(n)}{g(n)}) < \infty \Rightarrow f(n) \in \Theta(g(n))$
- $\lim_{n \rightarrow \infty} (\frac{f(n)}{g(n)}) > 0 \Rightarrow f(n) \in \Omega(g(n))$
- $\lim_{n \rightarrow \infty} (\frac{f(n)}{g(n)}) = \infty \Rightarrow f(n) \in \omega(g(n))$

Example Limit Proof

- By definition, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ means $\forall \varepsilon > 0, \exists n_0 > 0, \forall n \geq n_0, \frac{f(n)}{g(n)} < \varepsilon$.
- We can pick $c = \varepsilon$, then we have $f(n) < \varepsilon \cdot g(n), f(n) < c \cdot g(n), f(n) \in o(g(n))$.

Properties

- Reflexivity of $O, \Omega, \Theta : f(n) \in O(f(n))$
- Transitivity of all: $f(n) \in O(g(n)) \wedge g(n) \in \Theta(h(n)) \Rightarrow f(n) \in O(h(n))$
- Symmetry: $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
- Complementarity: $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n)), f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$

Useful Facts

- $e^x \geq 1 + x$
- For any $k > 0, a > 1, n^k \in o(a^n)$
- $n^i \in \Omega(\log^j n)$
- $\log_b a = \frac{\log_c a}{\log_c b}$
- $a^{\log_b c} = c^{\log_b a}$
- Stirling's Approximation: $n! = \sqrt{2\pi n} (\frac{n}{e})^n (1 + \Theta(\frac{1}{n}))$, $\log(n!) \in \Theta(n \log n)$
- $\sum_{k=1}^n k = \frac{1}{2}n(n+1) \in \Theta(n^2)$
- $\sum_{k=0}^n x^k = \frac{x^{n+1}-1}{x-1}$
- $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ when $|x| < 1$
- $H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$
- L'Hopital's: $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$

Solving Recurrences

Telescoping

- Goal: Make successive terms cancel out leaving first and last
- Mergesort: $T(n) = 2T(n/2) + n \Rightarrow \frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1$ and so on
- By expanding, we have $\log n$ layers which cancel out to leave $\frac{T(n)}{n} = \frac{T(1)}{1} + \log n$ and $T(n) = O(n \log n)$

Substitution

- Guess form of solution, then verify by induction
- e.g. Tower of Hanoi: Solve $T(n) = 2T(n-1) + 1$, guess $T(n) = 2^n - 1$, then verify
- Example: $T(n) = 4 \cdot T(n/2) + \sqrt{n}$
- Trying gives $T(n) \in O(n^3)$ and $T(n) \in \Omega(n)$, but nothing tight
- Guess $T(n) \in \Theta(n^2)$ using $T(n) \leq c_2 \cdot n^2 - c_1 \cdot n$ (to deal with the $\sqrt{n}$ term)
- For $c_1 = 1$, set c_2 large enough so $T(1) \leq c_2 - c_1$
- Upper Bound: $T(n) = 4 \cdot T(n/2) + \sqrt{n} \leq 4 \cdot (c_2 \cdot n^2/2^2 - c_1 \cdot n/2) + \sqrt{n} = c_2 \cdot n^2 - 2 \cdot c_1 \cdot n + \sqrt{n} \leq c_2 \cdot n^2 - c_1 \cdot n + (\sqrt{n} - c_1 \cdot n) \leq c_2 \cdot n^2 - c_1 \cdot n$
- Lower Bound: Guess $T(n) \geq c_3 \cdot n^2$ where $c_3 > 0$
- $T(n) = 4 \cdot T(n/2) + \sqrt{n} \geq 4 \cdot (c_3 \cdot n^2/2^2) + \sqrt{n} = c_3 \cdot n^2 + \sqrt{n} \geq c_3 \cdot n^2$
- Combining both gives $T(n) \in \Theta(n^2)$

Recursion Tree

- Draw recursion tree
- Multiply layers and sum of nodes in each layer

Master Theorem

- Solve recurrence of generic form $T(n) = aT(n/b) + f(n), a \geq 1, b \geq 1, f \in \Omega(1)$
- Classify work into splitting/combining and solving base cases
- Solving base cases is linear in the number of leaves, n^d
- Tree Height: $\log_b n$, Number of children of a node: a , Number of leaves: $a^{\log_b n} = n^{\log_b a} = n^d, d = \log_b a$ is the critical exponent
- Cost of splitting and combining is $f(n)$

- Floors and ceilings within recursive subproblem do not affect asymptotic growth of the function
- 1: $f(n) \in O(n^{d-\epsilon}) \Rightarrow T(n) \in \Theta(n^d), \epsilon > 0$
- 2: $f(n) \in \Theta(n^d \log^k n)$
 - 2a: $k \geq 0 \Rightarrow T(n) \in \Theta(n^d \log^{k+1} n)$
 - 2b: $k = -1 \Rightarrow T(n) \in \Theta(n^d \log \log n)$
 - 2c: $k < -1 \Rightarrow T(n) \in \Theta(n^d)$
- 3: $f(n) \in \Omega(n^{d+\epsilon}) \Rightarrow T(n) \in \Theta(f(n)), \epsilon > 0$, requires regularity condition $af(n/b) \leq cf(n), c < 1$ to ensure $f(n)$ is dominant term

$$af(n/b) \leq cf(n)$$

$$\frac{a}{c} f(n/b) \leq f(n)$$

$$(\frac{a}{c})^i f(1) \leq f(b^i)$$

$$\Omega(n^{d+\epsilon}) \ni n^{\log_b(\frac{a}{c})} f(1) = (\frac{a}{c})^{\log_b n} f(1) \leq f(n)$$

$$\epsilon = \log_b(\frac{a}{c}) - \log_b a > 0, n = b_i$$

Glossary

- Dominant Term: Largest term in time complexity
- Linearity: $f(x) \in \Theta(x)$, $f(x)$ is linear in x
- Polynomial: $f(n) = n^c$ for some $c \in \Theta(1)$, Polylog: $f(n) = (\log n)^c$ for some $c \in \Theta(1)$

Proof of Correctness

Iterative Algorithm

- Goal: Prove a given algorithm is correct
- Loop Invariant (Induction Hypothesis): Desirable conditions that should be satisfied at the start of each iteration
- Initialisation (Base Case): Show invariants are true at the start of the first iteration
- Maintenance (Inductive Step): Show that if invariant is true at the start of current iteration then it is also true at the start of the next iteration
- Termination (Induction implies Correctness): If the loop invariant holds at the end of the last iteration, then the algorithm outputs a correct answer

Dijkstra's Proof

- Loop Invariant: At the start of an iteration, $\forall u \in R, d(u) = \text{dist}(s, u)$ (R is processed vertices)
- Termination: At end of computation, $R = V$, and all computed distances are correct
- Maintenance: Claim for selected v , $\text{dist}(s, v) = \min_{u \in R} (d(u) + w(u, v))$
- Consider a shortest path P from s to v . Then all vertices in $P \setminus \{v\}$ must be in R
- If not, we should have taken x , the first vertex not in R along P , because $\min_{u \in R} (d(u) + w(u, v)) \geq \text{dist}(s, v) > d(y) + w(y, x) \geq \min_{u \in R} (d(u) + w(u, x))$
- Binary Heap: $O((m+n) \log n)$, Fibonacci Heap: $O(m + n \log n)$
- New Algorithm: $O(m \log^{2/3} n)$

Recursive Algorithms

- Base Case: Show correctness
- Inductive Step: Assume algorithm is correct for input size smaller than n , then show it is correct for size n

Binary Search Proof

- Base Case: Obvious
- Induct on array size, if mid is what we want, returns correctly, if not falls into a smaller and correct case

Divide and Conquer

- Divide problem into smaller subproblems
- Solve subproblems recursively
- Combine subproblem solutions to get the solution of the the full problem
- Naive Matrix Multiplication: $\Theta(n^3)$
- Divide and Conquer: $T(n) = 8T(n/2) + \Theta(n^2), T(n) \in \Theta(n^3)$, no improvement
- Strassen's Algorithm: $T(n) = 7T(n/2) + 18 \cdot n^2, T(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807...})$
- H-Index: Find greatest i such that $A[i] \geq i$, this can be done using binary search in $O(n)$ time
- However, we can improve this to $O(\log ans)$
- Try increasing 2^i where $i = 0, 1, 2, \dots$ until we get $i^* \geq ans$
- Then $i^*/2 < ans \leq i^*$ so we can binary search on $[1, i^*]$ with cost $O(\log i^*)$

Sorting

- Sort elements in an array in non-decreasing order
- Comparison Based: Elements can only be compared with each other and no other properties can be used
- Worst-case time complexity for comparison-based sorting is $\Omega(n \log n)$
- Decision Tree: Every node asks a question, and takes a child depending on answer, at leaf an output is given
- Model a comparison based algorithm as a decision tree, which has $n!$ leaves minimum, and height at least $\log(n!) \in \Theta(n \log n)$ by Stirling's
- To merge k sorted arrays of size n , $\Theta(kn \log k)$ is a tight lower bound
- The number of ways to combine is $\frac{(kn)!}{(n!)^k}$, so the complexity is $\Theta(kn \log k)$.
- Counting sort is not comparison-based, and finishes in $O(n+k)$ time
- Other Properties: Small running time (Worst / Average Case), Stability (Preserving original positions of equal items), In-Place (Uses $O(1)$ extra memory)

QuickSort Analysis

- $T(n) = T(j-1) + T(n-j) + cn$ if pivot is j th element
- Worst Case: $j = 1$ or $j = n$, $T(n) = T(n-1) + cn \in \Theta(n^2)$, proof by induction
- Average Case: $A(n)$ is expected running time when input is a uniformly random permutation
- $A(n) = \sum_{\pi} \frac{1}{n!}$ (running time of quick sort on π)
- $A(n) = \frac{1}{n} \sum_{j=1}^n [A(j-1) + A(n-j) + cn]$
- $A(n) = cn + \frac{2}{n} \sum_{j=0}^{n-1} A(j)$

- $n \cdot A(n) = cn^2 + 2 \sum_{j=0}^{n-1} A(j)$
- $(n-1) \cdot A(n-1) = c(n-1)^2 + 2 \sum_{j=0}^{n-2} A(j)$
- $n \cdot A(n) - (n-1) \cdot A(n-1) = c(2n-1) + 2A(n-1)$
- $n \cdot A(n) - (n+1) \cdot A(n-1) = n \cdot A(n) - (n-1) \cdot A(n-1) - 2A(n-1) = c(2n-1)$
- $\frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{c(2n-1)}{n(n+1)} < \frac{c(2n+2)}{n(n+1)} = \frac{2c}{n}$
- $\frac{A(n)}{n+1} < 2c \cdot (\frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{2}) + \frac{A(1)}{2}$ by telescoping
- $A(n) \in O(n \log n)$

Karatsuba's Algorithm

- Naive: To compute $C(x) = A(x) \times B(x)$ where they are polynomials, compute the coefficients of $C(x)$ in $O(n^2)$ with complete search
- Karatsuba's: Compute $A_1(x) \times B_1(x), A_2(x) \times B_2(x)$ still. However, instead of computing $A_1(x) \times B_2(x), A_2(x) \times B_1(x)$, instead compute $[A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)]$. Then $A_1(x) \times B_2(x) + A_2(x) \times B_1(x) = [A_1(x) + A_2(x)] \times [B_1(x) + B_2(x)] - A_1(x) \times B_1(x) - A_2(x) \times B_2(x)$
- Then we have $T(n) = 3 \cdot T(n/2) + O(n)$ which is $T(n) \in \Theta(n^{\log_2 3}) = \Theta(n^{1.58\dots})$

Randomised Algorithms

- Utilise randomisation to develop algorithms that are more efficient or simpler than their deterministic counterparts at the cost of allowing a small error probability
- Verification of Matrix Multiplication: Check if square $AB = C$, takes $O(n^{2.807})$ by Strassen's
- Freivald's: Choose a size n column vector where each entry is random binary number with uniform probability, then check if $ABv = Cv$ in $O(n^2)$ time
- If $AB = C$, then $ABv = Cv$ will output correctly
- If not, consider $C^* = AB - C, u = C^*v$. It will output an incorrect answer iff all $u_k = 0$
- Principle of Deferred Decisions: Instead of fixing all random choices in advance, we conceptually defer the outcomes of random choices until the moment they are actually needed
- Once we reveal random numbers excluding v_j , the non v_j term is fixed, and there is at most one choice of it which makes $u_i = 0$
- The probability of success is $\geq 1/2$, and can be increased to $\geq 1 - 1/2^t$ by repeating t times

Coupon Collector

- Given n types of coupons, we want to obtain all, how many must we buy to collect all types of coupons
- Throw m balls in n bins, and check the probability that every bin contains at least one ball
- For a specific bin, the probability it has no balls is $(1 - 1/n)^m = (1 - 1/n)^{n \cdot m/n} \leq e^{-m/n}$ using $1 + x \leq e^x$
- The probability at least one bin is empty is at most $n(1 - 1/n)^m \leq ne^{-m/n}$
- The probability is at most $1/n$ if $m \geq 2n \lceil \ln n \rceil$ since $e^{-2 \ln n} = \frac{1}{n^2}$
- Expected Value: $E[X] = \sum_x x \cdot Pr[X = x]$

- Markov Inequality: If X is a non-negative random variable and $a > 0$, then $Pr[X \geq a \cdot E[X]] \leq 1/a$, e.g. if the expected runtime is at most t , the runtime is at most $100 \cdot t$ with probability at least 0.99
- Linearity of Expectation: $E[A + B] = E[A] + [B]$
- Indicator Random Variable: Let ϵ be an event, then the indicator random variable 1_ϵ is defined as 1 if it occurs or 0 otherwise, so $E[1_\epsilon] = Pr[\epsilon]$

Quicksort Again

- Let $X_{i,j}$ be the number of comparisons made between a_i and a_j
- $E[\text{comparisons}] = E[\sum_{i < j} X_{i,j}] = \sum_{i < j} E[X_{i,j}]$
- Let $\epsilon_{i,j}$ denote the event $X_{i,j} = 1$
- We claim $Pr[\epsilon_{i,j}] = \frac{2}{j-i+1}$ for $i < j$
- A comparison is made between them only if the pivot chosen is either one of them, and they must start in the same array
- Each number is equally likely to be the pivot, and from $(a_i, \dots, a_j)$, there are $j - i + 1$ elements
- $E[\text{comparisons}] = \sum_{i,j} \frac{2}{j-i+1} = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j-i+1} = 2 \sum_{i=1}^{n-1} (\frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n-i+1}) \in O(n \log n)$
- Randomised quick sort finishes in $O(n \log n)$ time with probability at least 0.99

Summary of Techniques

- Las Vegas: Always correct, but time complexity guarantee is only in expectation
- Monte Carlo: Output is correct with some probability, but the time complexity guarantee always holds
- Principle of Deferred Decision: If we can show $\forall x, Pr[\epsilon|X = x] \geq p$, then $Pr[\epsilon] \geq p$
- Success Probability Amplification: The error probability can be reduced to at most f by repeating the algorithm $\lceil \log \frac{1}{f} \rceil$ times
- Union Bound: Upper bound the probability that bad ϵ occurs, and you know $Pr[\epsilon] = Pr[\epsilon_1 \vee \dots \vee \epsilon_n] \leq Pr[\epsilon_1] + \dots + Pr[\epsilon_n]$
- Then to make sure $Pr[\epsilon] \leq f$, we just need to make sure $Pr[\epsilon_i] \leq \frac{f}{n}$ for each i
- Markov Inequality: If X is a non-negative random variable and $a > 0$, then $Pr[X \geq a \cdot E[X]] \leq \frac{1}{a}$
- Can be used to turn Las Vegas algorithm into Monte Carlo algorithm via Markov inequality
- Linearity of Expectation: $E[A + B] = E[A] + E[B]$

Transmission

- Goal: Check whether two n-bit strings are equal
- Method: Create a set S of n^2 smallest primes
- Sample a random prime p from S
- Send p and $S_A \bmod p$
- Check $S_B \bmod p$ against $S_A \bmod p$
- When they are equal, the output will be correct
- When they are not equal, the output is incorrect iff p divides $|S_A - S_B|$, let this value be d
- Then $1 \leq d < 2^n$

- d has at most $n - 1$ distinct prime factors, so the probability p is a divisor of d is at most $\frac{n-1}{|S|} = \frac{n-1}{n^2} = \frac{n}{n^2} - \frac{1}{n^2} = \frac{1}{n} - \frac{1}{n^2} \leq \frac{1}{n}$
- Any correct algorithm requires $\Omega(n)$ bits by decision trees

Partition of Graph

- Partition a graph randomly by assigning each vertex in the set V_1 or V_2 to get the expected number of edges crossing to be $\frac{|E|}{2}$
- To improve slightly, pick one specific edge and force it to cross, then we get $\frac{|E|+1}{2}$ which is marginally better
- If $|V|$ is even, we can select V_1 uniformly at random from all size $|V|/2$ subsets of V and setting V_2 as the complement
- $Pr(u \in V_1) = \frac{1}{2}$
- $Pr(v \in V_2|u \in V_1) = 1 - Pr(v \in V_1|u \in V_1) = 1 - \frac{|V|/2-1}{|V|-1} = \frac{|V|/2}{|V|-1}$
- We can sum the symmetric case to get $E[X_e] = 2 \cdot Pr(u \in V_1, v \in V_2) = 2 \cdot Pr(u \in V_1) \cdot Pr(v \in V_2|u \in V_1) = 2 \cdot \frac{1}{2} \cdot \frac{|V|/2}{|V|-1} = \frac{|V|/2}{|V|-1}$
- By linearity of expectation, $E[X] = \sum_{e \in E} E[X_e] = |E| \times \frac{|V|/2}{|V|-1} = \frac{|E|}{2} \times \frac{|V|}{|V|-1}$

Dynamic Programming

- Divide problem into smaller overlapping subproblems
- Solve the subproblems recursively with memoisation
- Combine the subproblem solutions to get the solution of the full problem
- Bottom Up: Compute the values in the array in some order, starting from base cases

LCS

- C is a subsequence of A if C can be obtained by removing elements from A
- LCS: Given two arrays, output a longest common subsequence C , or equivalently, find a largest non-crossing matching
- Brute Force: Check all possible subsequences of A and see if they fit in B , takes $O(m \cdot 2^n)$ time
- Recursive Approach: Base cases: $i = 0$ or $j = 0$
- Inductive Step: Compute $LCS(n, m)$ given $LCS(n-1, m), LCS(n, m-1), LCS(n-1, m-1)$
- If $a_n \neq b_m$, take the longer of $LCS(n-1, m), LCS(n, m-1)$
- If $a_n = b_m = x$, then take $LCS(n-1, m-1) \circ x$
- Use cut and paste to prove observation for equality case
- Show $LCS(n, m)$ should terminate with a_n when $a_n = b_m$
- Suppose not and it ends with something else.
- We can then append a_n and get a longer subsequence, which is a contradiction.
- Optimal Substructure: An optimal solution to a problem can be constructed from optimal solutions to its subproblems
- With memoisation, takes $O(mn)$ time

Knapsack

- Pack a set of items into a container with a capacity constraint to maximise the total value
- Formally, output a subset $S \subseteq [n]$ such that $\sum_{i \in S} w_i \leq W$ maximises $\sum_{i \in S} v_i$
- Suppose S is an optimal solution; if $n \in S$, then $S \setminus \{n\}$ is an optimal solution to the subproblem of n with $W - w_n$
- If $n \notin S$, then S is an optimal solution to the subproblem without n
- If not, a better solution can be obtained by cut and paste, by replacing S with an optimal solution to the subproblem
- In the first case, we have $m[n, W] = m[n-1, W - w_n] + v_n$, and in the second case, we have $m[n, W] = m[n-1, W]$

Change-Making Problem

- Find the minimum number of coins of specific k denominations that add up to n cents
- $M[n] = 1 + \min_{i \in [k]} M[n - d_i]$

Cut and Paste

- Assume there is an optimal solution to S to some problem instance
- Look at the sub solution to S that corresponds to some smaller problem
- If the sub-solution were not the optimal one for the subproblem, we could cut it out, and paste in the truly optimal solution for the sub problem
- This produces a new solution which has strictly greater value
- This contradicts the assumption that S was optimal unless the sub-solution was also optimal

Greedy Algorithms

- Recast the problem so that only one subproblem needs to be solved at each step

Fractional Knapsack

- Knapsack but we can take fractions of items
- Optimal Substructure: If we remove y kg of item j fro the optimal knapsack, then the remaining load must be optimal for weight at most $W - y$ kg from the remaining $n - 1$ items and $w_j - y$ kg of item j
- Cut and Paste: Let X be the value of the optimal knapsack, and suppose that the remaining load after removing y of item j was not the optimal knapsack for $W - y$ from $n - 1$ items and $w_j - y$ of item j
- Then there is a knapsack with value $> X - v_j \cdot \frac{y}{w_j}$ with weight $\leq W - y$ among $n - 1$ other items and $w_j - y$ of item j
- Combining y of j gives knapsack with value $> X$ and weight $\leq W$ which is a contradiction
- So there must be optimal substructure.
- Greedy Choice Property: Let j^* be the item with maximum v_j/w_j . Then there exists an optimal knapsack containing $\min(w_{j^*}, W)$ of item j^*
- Suppose an optimal knapsack has $x_{j^*} < \min(w_{j^*}, W)$. From the rest of the knapsack, pick y such that $\sum y = \min(w_{j^*}, W) - x_{j^*}$. Then we can replace these

- by $\min(w_{j^*}, W) - x_{j^*}$ of item j^* , then total value does not decrease since j^* has maximum ratio
- Greedy Algorithm: Use the greedy choice property to take as much j^* as possible, then repeat until we run out of weight, $O(n \log n)$

Greedy Algorithm Paradigm

- Cast the problem where we have to make a choice and are left with one subproblem to solve
- Prove (using exchange argument) that there is always an optimal solution to the original problem that makes the greedy choice so the greedy choice is safe
- Use optimal substructure (proof by contradiction or cut-and-paste argument) to show that we can combine an optimal solution to the subproblem with the greedy choice to get an optimal solution to the original problem

Huffman Code

- Binary Coding: Mapping from alphabet set to binary string
- Fixed Length Coding: Each alphabet is a binary string of fixed length, $m \lceil \log_2 n \rceil$ bits needed for m alphabets with alphabet size n
- Letter Frequency Variation: Give more frequent alphabets encoding with shorter bit strings
- Average Bit Length: $ABL(\gamma) = \sum_{x \in A} f(x) \cdot |\gamma(x)|$
- Need to be careful of ambiguity caused when the encoding of one letter is a prefix of another
- Prefix Coding: A coding where the above ambiguity does not exist
- Problem: Find a prefix coding γ where $ABL(\gamma)$ is the minimum
- Huffman Coding: Labeled binary tree where each child is either a 0 or 1 with a frequency, and leaf nodes represent alphabets
- For each prefix code of a set of alphabets, there exists a binary tree where there is a bijective mapping between the alphabets and leaves, and the label of a path from root to leaf corresponds to the prefix code of the corresponding alphabet at the leaf
- $ABL(\gamma) = \sum_{x \in A} f(x) \cdot \text{depth}_T(x)$
- Observation 1: The ideal binary tree should be a full binary tree where every internal node has degree 2, if not we have redundant nodes
- More frequent alphabets should be closer to the root
- Let $a_1, a_2, \dots, a_n$ be the alphabet in non-decreasing order of frequency (a_1 is least frequent)
- Observation 2: The least frequent alphabet a_1 cannot be present at a higher level than the deepest one, since swapping it with a lower one cannot increase ABL
- Observation 3: For a_2 , it can be the sibling of a_1
- Lemma: There exists an optimal prefix coding in which a_1, a_2 appear as siblings (not necessarily in all!)
- Observation 4: For siblings a_1 and a_2 , we can merge them into a new $f(a')$
- $OPT_{ABL}(A)$: Minimum ABL over all prefix codes for alphabet A , $OPT(A)$: Prefix code for alphabet A with $ABL = OPT_{ABL}(A)$
- Let $A' = a_3, \dots, a', \dots, a_n$ be $n - 1$ alphabets with $f(a') = f(a_1) + f(a_2)$

- We claim that $OPT_{ABL}(A) = OPT_{ABL}(A') + f(a_1) + f(a_2)$, and this implies the algorithm is optimal
- Algorithm
 - If $|A| = 2$, return a single node with a_1, a_2 as children
 - If not, repeatedly remove a_1, a_2 from A
 - Create letter a' and calculate $f(a') = f(a_1) + f(a_2)$
 - Insert a' into A
 - Update tree to $OPT(A)$
 - Replace leaf node a' in tree with node containing children a_1, a_2
 - Heap for alphabet and frequencies is $O(\log n)$ insertion and removal, overall $O(n \log n)$
- Now we need to prove $OPT_{ABL}(A) = OPT_{ABL}(A') + f(a_1) + f(a_2)$
- Let T' be the optimal binary tree for $OPT_{ABL}(A')$, then construct T by replacing a'
- This gives $ABL = OPT_{ABL}(A') + f(a_1) + f(a_2)$, with proof below
 - Let d be the depth of a' in T' with ABL $OPT_{ABL}(A')$
 - For the tree we constructed for A , we have $ABL =$

$$\begin{aligned}
 &= \sum_{i=1}^n f(a_i) \cdot \text{depth}_{T'}(a_i) \\
 &= \sum_{i=3}^n f(a_i) \cdot \text{depth}_{T'}(a_i) + f(a_1) \cdot (d+1) + f(a_2) \cdot (d+1) \\
 &= \sum_{i=3}^n f(a_i) \cdot \text{depth}_{T'}(a_i) + (f(a_1) + f(a_2)) \cdot d + f(a_1) + f(a_2) \\
 &= \sum_{i=3}^n f(a_i) \cdot \text{depth}_{T'}(a_i) + f(a') \cdot \text{depth}_{T'}(a') + f(a_1) + f(a_2) \\
 &= \sum_{i=3}^n f(a_i) \cdot \text{depth}_{T'}(a_i) + f(a') \cdot \text{depth}_{T'}(a') + f(a_1) + f(a_2) \\
 &= OPT_{ABL}(A') + f(a_1) + f(a_2)
 \end{aligned}$$

- For T' corresponding to $OPT_{ABL}(A')$, splitting a' gives a coding with $ABL = OPT_{ABL}(A') + f(a_1) + f(a_2)$, and thus $OPT_{ABL}(A) \leq OPT_{ABL}(A') + f(a_1) + f(a_2)$ by construction
- For T corresponding to $OPT_{ABL}(A)$, replacing a_1, a_2 with a' gives $ABL = OPT_{ABL}(A) - f(a_1) - f(a_2)$, and implies $OPT_{ABL}(A') \leq OPT_{ABL}(A) - f(a_1) - f(a_2)$ by construction as well
- Combining them gives the recurrence we want by showing equality